AI Engineering and Claude Architect · Part I, Foundations

Working With Claude, Lesson 4

Thinking, Effort and Model Selection

You migrate to Opus 5.5, and suddenly your request returns a 400. You remove the rejected parameter, and now answers feel shallower than they did on the previous model. Both changes have the same explanation: the controls for reasoning moved, and so did their defaults.

Module	AI03 of 13
Lesson	4 of 6
Prerequisites	AI02 L7, AI03 L1
Exam relevance	Low. The guide mentions extended thinking only in TS 4.6, saying that it's less effective than an independent review. Effort isn't in the guide. Critical for real systems
Reading time	~30 minutes

Contents

1. Thinking, Effort and Model Selection

Lesson 4 of 6 · Working With Claude

Thinking, Effort and Model Selection

Key idea

On current Claude models, you no longer budget thinking tokens or set temperature. You choose a model and an effort level, and Claude decides how much to think. Thinking is billed whether or not you can see it, and defaults differ by model. **So set effort explicitly for each route, test it on your evals, and never assume that a parameter that worked on one model will work on the next.**

> *Before: model tiers and cost per task (AI02 L7), the request format (AI03 L1). Teaches: adaptive thinking, effort, display modes, what changed across models, and how to migrate safely. Used by: AI06 (agents), AI08 L1, AI10 L2, AI10 L5.*

The hook: the failed migration

A team moves a working request to `claude-opus-5-5`:

```
{
  "model": "claude-opus-5-5",
  "max_tokens": 16000,
  "temperature": 0.2,
  "thinking": {"type": "enabled", "budget_tokens": 8000},
  "tool_choice": {"type": "any"},
  "messages": [{"role": "assistant", "content": "{"}]
}
```

Each of these settings was reasonable at one time. On this model, each of them fails.

Setting	Result on Opus 5.5	Replacement
`temperature: 0.2`	400	None needed. Use `effort` for depth
`thinking.budget_tokens`	400	Omit `thinking`, or send `adaptive`. Use `effort`
`tool_choice: any`	400	`auto` with an instruction and `strict`, or structured outputs
Assistant prefill	400	Structured outputs or prompt instructions

And after removing them, there's a subtler issue. **Opus 5.5's default effort is** `medium`, one level below Opus 5's default of `high`. So a straight migration can feel shallower, unless you set effort explicitly.

> *Version-sensitive*
>
> This lesson's model details come from Anthropic's API reference as of October 2026. **This is the fastest-changing material in the course. Check the current docs when you act on it.**

Intuition: a contractor's scope, not their hours

The old `budget_tokens` was like telling a contractor, "Spend exactly eight hours thinking about this." Too low, and they rush. Too high, and you pay for idle time.

`effort` is more like agreeing on a scope: "This needs a quick look" versus "Treat this as critical." The contractor decides how long each part really takes. Adaptive thinking works on that principle: you state how much the result matters, and Claude allocates its reasoning.

Adaptive thinking

Extended thinking lets Claude reason before and between its responses. **Adaptive thinking** is the current form, where Claude decides dynamically when and how much to think, and can think between tool calls.

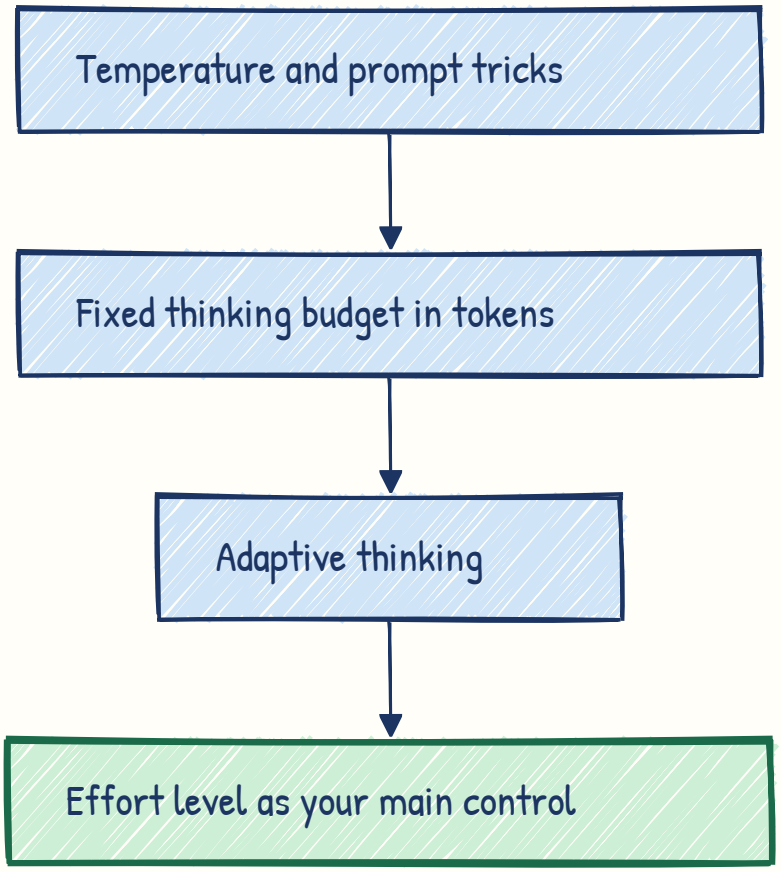


Figure 1. The evolution of reasoning controls. Each step gave the model more of the decision, and left you with a simpler dial.

Per-model rules

Model	`thinking`	Omit it, and	`effort` default
Fable 5.1	`adaptive`, or omit. `disabled` returns 400	Runs adaptive, always on	`high`
Opus 5.5	`adaptive`, or omit. `disabled` returns 400	Runs adaptive	**`medium`**
Opus 5	`adaptive`, or `disabled` at `high` or below	Runs adaptive	`high`
Opus 4.8, 4.7	`adaptive` or `disabled`	**Runs without thinking**	`high`
Sonnet 5.5	`adaptive`, or omit. To turn thinking off, send `between_tools`	Runs adaptive	`high`, with recalibrated levels
Haiku 5.5	`adaptive`, or `disabled` at `high` or below	Runs adaptive	**`medium`**

The "omit it" column is the trap: **the same code means different things on different models.**

What you see versus what you pay

`display`	You receive	You're billed for thinking?
`omitted` (default on current models)	`thinking` blocks with empty text	Yes
`summarized`	A readable summary	Yes
`updates` (beta)	Short progress notes between tool calls	Yes

The raw chain of thought is never returned. And since thinking counts as output tokens even when it isn't shown, **a response with short visible text can still have a surprisingly large `output_tokens` value.**

For chat UIs: the default `omitted` looks like a long pause before output starts. Set `display: "summarized"` if you stream reasoning to users.

Effort

`effort` goes inside `output_config`, not at the top level. It controls thinking depth and overall token spend.

Level	Reasonable starting use	Notes
`low`	Chat, classification, high-volume routes, simple subagents	Fewer tool calls, terser
`medium`	A cost-saving step-down where quality holds	Default on Opus 5.5 and Haiku 5.5
`high`	Intelligence-sensitive work	Often a good balance
`xhigh`	Hard coding and long agentic work	
`max`	Correctness matters more than cost	Raise only when measurements show headroom

These starting points follow Anthropic's reference. The same reference warns that **which workloads repay higher effort is a property of the workload**, so measure on real requests and tune per route, instead of globally.

Java, from `code/ai03-claude/.../ModelRequests.java`, compiled against `anthropic-java` 3.2.0 but not run:

```java
public static MessageCreateParams hardReasoning(String prompt) {
    return MessageCreateParams.builder()
            .model("claude-opus-5-5")
            .maxTokens(16_000L)
            .thinking(ThinkingConfigAdaptive.builder()
                    .display(ThinkingConfigAdaptive.Display.SUMMARIZED)
                    .build())
            .outputConfig(OutputConfig.builder()
                    .effort(OutputConfig.Effort.HIGH)
                    .build())
            .addUserMessage(prompt)
            .build();
}
```

Should you switch model, or lower effort?

Anthropic's cost guidance is direct: before building a multi-model cascade, **measure the most capable model at lower effort.** Lower effort on the newest models often matches or exceeds an older model at high effort, and one model means one cache namespace. Judge cost per completed task, not per request, since a cheaper request that needs more retries isn't cheaper.

Passing thinking back

Thinking blocks in your history must be returned unchanged, including after a model switch. The API drops any that the current model can't read, and doesn't bill for them. This is another reason for AI03 L1's rule to append the whole response.

Preserved thinking adds a strict rule on some models: editing earlier turns invalidates thinking blocks. Newer accounts get a 400 on edited history. So a harness that rewrites old messages, for instance to trim tool results, may need a redesign (AI03 L6).

Other controls worth knowing

Feature	What it does	Status and caution
Task budgets	Give Claude a token ceiling across an agent loop, so that it paces itself	Beta. Advisory, and different from `max_tokens`
Fast mode	Up to 2.5× output speed at premium pricing	Research preview, on some Opus models only
Refusal fallbacks	Route a declined request to another model	Beta server-side parameter on supported models
Priority Tier	Capacity commitment	Not available on every current model

How Anthropic approaches it

Problem	Anthropic's change	Trade-off
Token budgets were hard to tune	Adaptive thinking	Less direct control over thinking size
Sampling settings were overused	Removed on current API version	No more temperature knob
Reasoning could be extracted	The raw chain of thought is never returned	You debug from outputs and summaries
Workloads differ	`effort` levels	You must evaluate per route
Harnesses edited history	Preserved thinking	Append-only design

What this shows about the provider relationship: model upgrades can be breaking changes. Plan for them the way you'd plan a major framework upgrade.

The production architecture

This is a recommended reference design, not a specific company's system.

Baseline: parameters scattered through code. Each service builds requests in its own way, so a migration means finding them all, and each one fails differently.

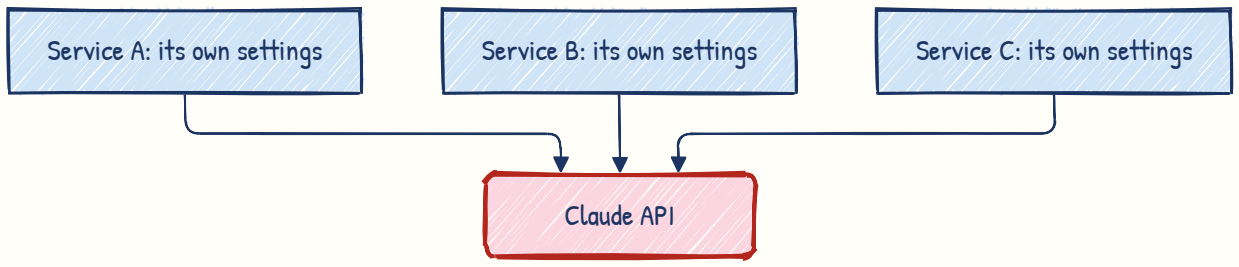


Figure 2. Baseline. Every service holds its own request settings, so one model change becomes a hunt across the codebase.

Improved: one config per route. A central table maps routes to model and effort. New problem: **each model accepts different parameters, so a valid config for one model is a 400 on another.**

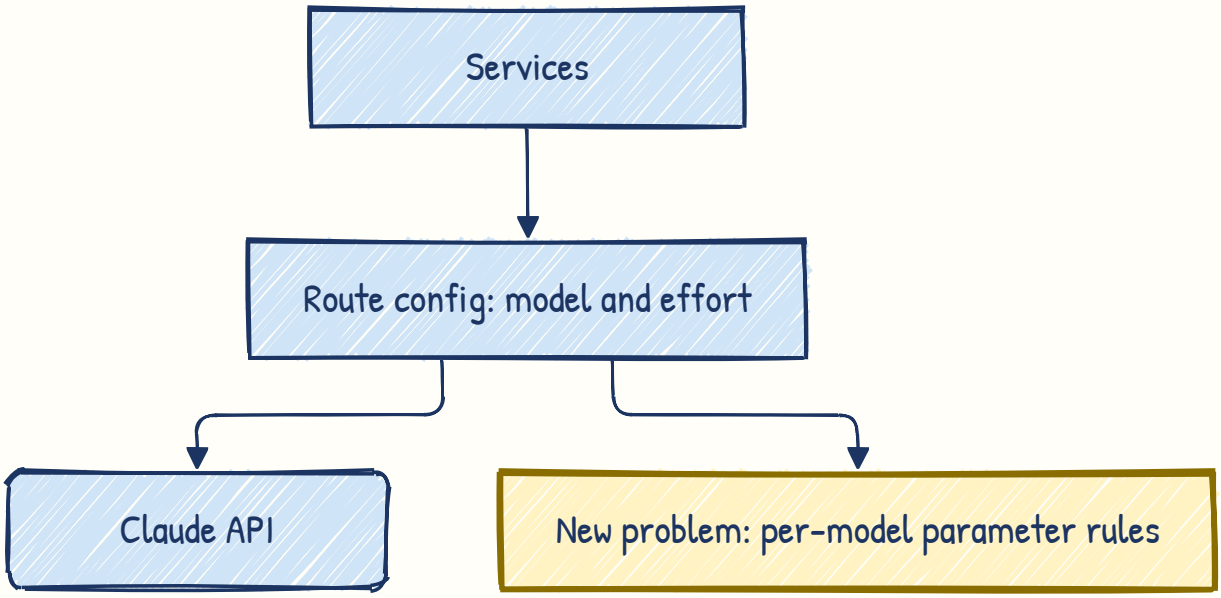


Figure 3. Improved. Settings live in one place, but nothing yet stops a config from sending parameters its model rejects.

Production: a request policy per model, validated before release.

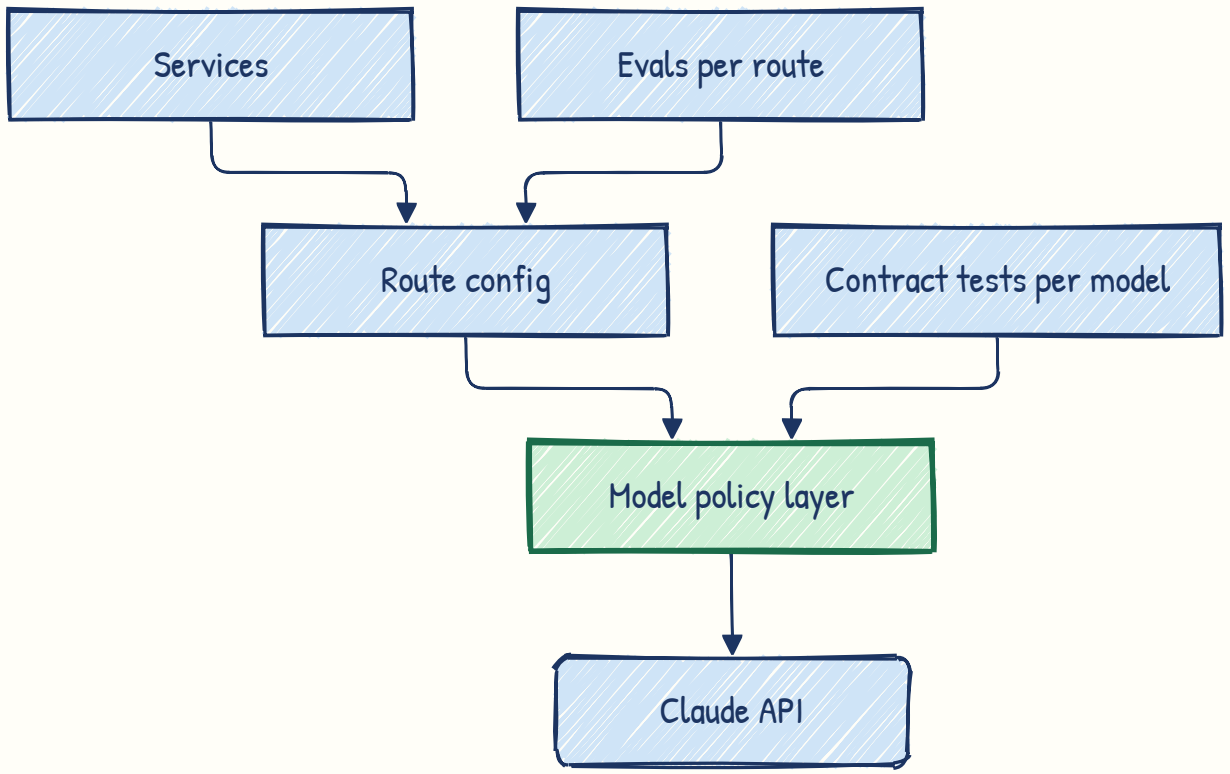


Figure 4. Production. Route config says what you want, and a model policy knows what each model accepts. Contract tests run against each model, and evals gate every change.

Component	Why it exists	How it communicates
Route config	Records the decision: this task uses this model at this effort	Config service or versioned file
Model policy layer	Translates intent into parameters that the model accepts	In the AI gateway (AI10 L2)
Contract tests	Catch 400s before production	CI, using small real requests
Evals	Confirm that quality holds after any change	CI (AI10 L3)

Migration checklist

Step	Why
1. Recount tokens on the new model	Tokenizers can differ (AI02 L1)
2. Remove rejected parameters	Sampling, budgets, prefill, forced tools on some models
3. Set `effort` explicitly	Defaults differ
4. Handle `refusal` and any new stop reasons	New safety classifiers
5. Review prompts	Older prompts can be over-prescriptive
6. Run evals and compare cost per task	Quality and cost both shift
7. Canary, with rollback	Limits damage

Failures

Failure	Containment
400 after upgrade	Contract tests catch it before release
Output tokens jump	Thinking is billed. Lower effort, and alert on spend
Long turns hit timeouts	Stream, and plan for long-running requests
Fallback model behaves differently	Test the fallback path with evals

Trade-offs

Decision	Gain	Cost
Higher effort	Better on hard tasks	More tokens and latency
Lower effort	Cheaper and faster	May fall below your quality bar
`display: summarized`	Visibility for users and debugging	Summaries aren't the raw reasoning
Effort per route	Fits each workload	More configuration and evaluation
One model across routes	Simple, with one cache namespace	May overpay on trivial tasks

When not to tune this: at low volume, set a sensible effort, and spend your time on evals and prompts.

Common mistakes

1. **Assuming that omitting `thinking` means off.** It depends on the model.
2. **Assuming default effort stayed the same.** Opus 5.5 and Haiku 5.5 default to `medium`.
3. **Assuming hidden thinking is free.** It's billed.
4. **Putting `effort` at the top level.** It belongs in `output_config`.
5. **Swapping model IDs with no tests.** Parameters and behavior both change.
6. **Using thinking in place of independent review.** TS 4.6 specifically says independent instances catch more.

What to say in an interview

"On current models I don't tune temperature or thinking budgets. I choose a model and effort per route, and validate on evals. I'd keep per-model parameter rules in a gateway, because the same request can be a 400 on another model. And I'd treat a model upgrade like a framework upgrade: recount tokens, set effort explicitly, run contract tests and evals, then canary."

Certification lens

How this lesson maps to CCAR-F

Almost nothing here is tested directly. The one tested point is in **TS 4.6**: an independent review instance is more effective than self-review instructions or extended thinking. Note that the guide predates some of these API changes, so it won't test `effort`.

These are original practice questions, not real or leaked exam questions.

Q Q1 · Exam style, scenario: Claude Code for Continuous Integration · Medium. Claude generates a module, and reviewing it in the same session misses subtle bugs. Someone suggests enabling extended thinking for the review. What does the exam guide suggest is more effective?

A. Add "be extra critical of your own code" to the prompt.

B. Enable extended thinking in the same session.

C. Run the review in an independent instance, without the generator's reasoning context.

D. Have the same session review it twice.

A **C** (TS 4.6). The generating session retains its reasoning, making it less likely to question its own decisions. A, B and D all keep that context.

Q Q2 · Error identification · Medium. Name every problem with this request to `claude-opus-5-5`: `temperature: 0`, `thinking: {"type": "disabled"}`, and `effort: "high"` as a top-level field.

A (1) `temperature` isn't accepted. (2) `disabled` returns a 400 on Opus 5.5, so omit it and lower effort instead. (3) `effort` belongs inside `output_config`.

Q Q3 · Performance and cost · Hard. A classification route on Opus 5.5 is too expensive, and someone proposes a three-model cascade. What do you test first, and why?

A The same model at `low` effort. Anthropic's guidance suggests measuring this first, since lower effort can hold quality, and a single model keeps one cache namespace with no routing logic. Compare cost per completed task on an eval set. Then, if it isn't enough, try cheaper models.

Revision - Lesson 4

→ Current models use adaptive thinking and effort, not budgets or temperature.

→ Effort goes in output_config: low, medium, high, xhigh, max.

→ Defaults differ: Opus 5.5 and Haiku 5.5 use medium. Most others use high.

→ Omitting thinking means different things on different models.

→ Thinking is billed even when hidden. The raw chain of thought is never returned.

→ Pass thinking blocks back unchanged. Editing history can invalidate them.

→ Rejected on Opus 5.5: temperature, budget_tokens, disabled thinking, forced tool_choice, prefill.

→ Try lower effort before building a model cascade.

→ Treat a model upgrade as a breaking upgrade: recount, set effort, contract tests, evals, canary.

→ For the exam: an independent instance beats extended thinking for review.

Your notes